

MU5MAI04

Approfondissement C++

Daphné Giorgi

Table of contents

Description	3
Objectifs	3
Evaluation	3
I GIT ET GITHUB	4
Objectifs	5
1 Description	6
2 Concepts de base	7
2.0.1 Espaces de travail	7
2.0.2 Commandes	7
3 Les remote	9
4 Install et configuration	10
4.0.1 Install de git	10
4.0.2 Configuration	10
5 Initialiser un projet	12
5.0.1 Initialisation d'un répertoire	12
5.0.2 Clone d'un projet distant	13
6 Travail en local	14
7 Travail avec remote	15
8 Travail en équipe	16
9 Workflow	17
10 .gitignore	18
11 Evaluation	20

Description

Ce cours concerne les étudiants IMPA et IMPC du [Master Mathématiques et Applications](#).

Emplois du temps : [EdT IMPA](#) et [EdT IMPC](#).

Moodle : [MU5MAI04](#)

Organisation : Chaque séance mélangera cours et travaux pratiques.

Prérequis : Fondamentaux de C++ par V. Lemaire.

Objectifs

Savoir développer un projet en C++ en utilisant les outils pour le développement tels que

- gestion de version
- debug
- test
- intégration continue
- notebook

Evaluation

La note finale sera donnée sur la base d'un travail à effectuer en binôme (deux personnes), au cours du S1, pendant les séances du cours.

On rappelle les règles de fonctionnement des binômes :

- pas de binôme mixte (apprenti - non apprenti)
- pas plus de trois projets au cours de l'année avec la même personne.

Chaque binôme devra rendre un **lien vers un dépôt git** contenant l'historique des commit du projet (et donc aussi la version finale du code). On ne vous demande pas de rapport. Une discussion d'environ **trente minutes** aura lieu pour m'aider à juger ce que vous avez fait. La date de rendu du dépôt git sera une semaine avant la soutenance.

Part I

GIT ET GITHUB

Objectifs

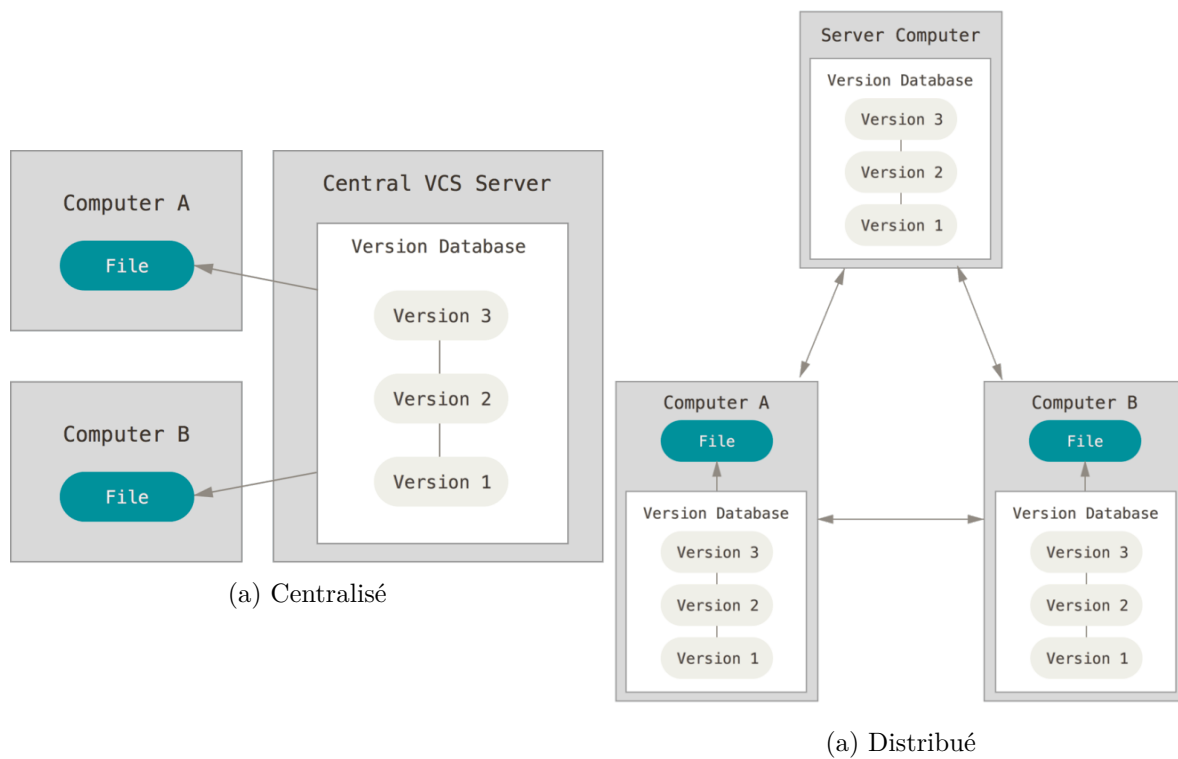
- Gérer un projet C++ avec Git
- Travailler en équipe avec GitHub
- Utiliser les branches, PR et git hooks

Sommaire :

[Chapter 1](#) [Chapter 2](#) [Chapter 3](#) [Chapter 4](#) [Chapter 5](#) [Chapter 6](#) [Chapter 7](#) [Chapter 8](#) [Chapter 9](#)
[Chapter 10](#)

1 Description

`git` est un système de contrôle de version distribué, libre, conçu pour être rapide et efficace.



Il est utilisé pour

- tracer les modifications dans un projet,
- identifier par qui ont été faites ces modifications,
- coder en équipe.

2 Concepts de base

2.0.1 Espaces de travail

Workspace/dossier de travail : Le dossier courant, ce que je vois lorsque j'ouvre un fichier et je travaille dessus.

Repository/Dépôt : Un dossier dans lequel Git enregistre le projet et son historique.

- **Local repository/dépôt local** : Endroit sur mon ordinateur où `git` stocke l'information sur l'historique des fichiers contenus dans le projet.
- **Remote repository/dépôt distant** : Endroit où est centralisé l'historique de mon projet, souvent sur un cloud de type GitHub/GitLab/..., et où tous les collaborateurs peuvent contribuer.

Staging area/index : Espace de stockage temporaire de l'historique `git`, endroit de *pre-commit*.

2.0.2 Commandes

Navettes qui permettent de se déplacer entre les espaces.

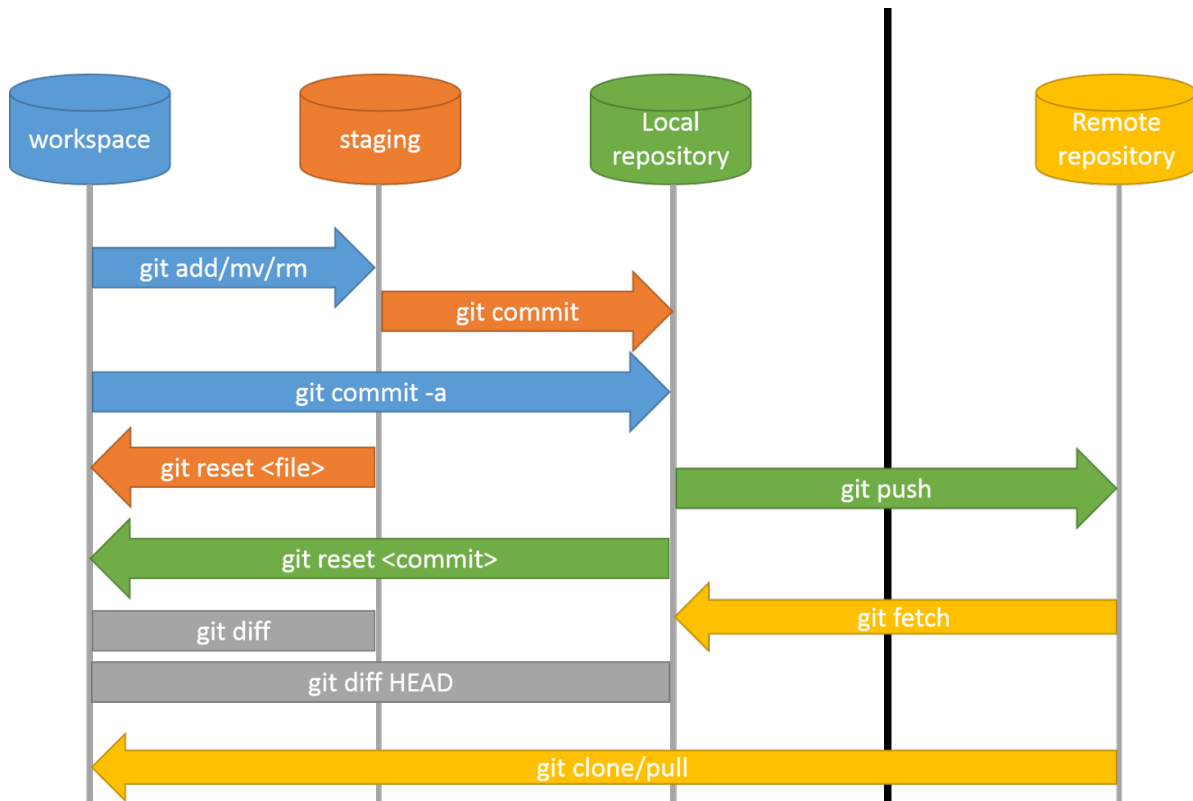


Figure 2.1: Répertoires locaux et distants

clone/Cloner : Copie d'un dépôt distant sur votre ordinateur.

stage/Index : Indiquer à Git les modifications que vous souhaitez enregistrer ensuite.

commit : Sauvegarde des modifications effectuées, ajout d'une ligne dans l'historique du projet.

branch/Branche : Travail sur différentes versions ou fonctionnalités en même temps.

merge/Fusionner : Combiner des modifications provenant de différentes branches.

pull/Récupérer : Obtenir les dernières modifications d'un dépôt distant.

push/Pousser : Envoyer vos modifications à un dépôt distant.

3 Les remote

[GitHub](#) n'est pas la même chose que `git`.

GitHub, tout comme GitLab, Bitbucket, ou autres, est un hébergeur de répertoires `git`.

Exercice

Création d'un compte sur [GitHub](#).

4 Install et configuration

4.0.1 Install de git

Vous pouvez télécharger Git gratuitement à partir de git-scm.com.

- **Windows** : Téléchargez et exécutez le programme d'installation.
Cliquez sur « Suivant » pour accepter les paramètres recommandés.
Cela installera Git et Git Bash.

- **macOS** : Si vous utilisez Homebrew, ouvrez Terminal et tapez

```
brew install git
```

Ou téléchargez le fichier `.dmg` et glissez Git dans votre dossier **Applications**.

- **Linux** : Ouvrez votre terminal et utilisez votre gestionnaire de paquets.

Par exemple, sur Ubuntu :

```
sudo apt-get install git
```

4.0.2 Configuration

La toute première fois, il est nécessaire de configurer `git` pour lui indiquer qui nous sommes.

Cette configuration peut être changée à tout moment et affecte seulement le nom et le mail qui apparaissent dans le commits.

User name

Tapez la commande

```
git config --global user.name "Prénom Nom"
```

email

Tapez la commande

```
git config --global user.email "monadresse@exemple.com"
```

Raccourcis

Parfois, taper les commandes `git` peut paraître long pour une utilisation régulière.

Il est possible de configurer des raccourcis, par exemple, en tapant

```
git config --global alias.ci commit
```

la commande `git ci` sera équivalente à `git commit`.

Pour ouvrir le fichier de config en mode édition on peut également taper

```
git config --global --edit
```

Exercice

Installation et configuration de git en local.

5 Initialiser un projet

On peut démarrer un dépôt `git` de deux manières :

- transformer un répertoire existant en dépôt `git`,
- cloner un répertoire distant.

Dans les deux cas, un répertoire local sur lequel travailler sera présent sur votre machine, on le reconnaît par la présence, à sa racine, d'un sous-répertoire `.git`.

5.0.1 Initialisation d'un répertoire

Partons la structure de code suivante :

```
CalculatorProject/  
  src/  
    Calculator.cpp  
  include/  
    Calculator.h  
  README.md
```

avec

```
include/Calculator.h
```

```
#ifndef CALCULATOR_H  
#define CALCULATOR_H  
  
class Calculator {  
public:  
    double add(double a, double b);  
    double subtract(double a, double b);  
    double multiply(double a, double b);  
    double divide(double a, double b);  
};  
  
#endif
```

src/Calculator.cpp

```
#include "Calculator.h"
#include <stdexcept>

double Calculator::add(double a, double b) {
    return a + b;
}

double Calculator::subtract(double a, double b) {
    return a - b;
}

double Calculator::multiply(double a, double b) {
    return a * b;
}

double Calculator::divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("Division by zero");
    return a / b;
}
```

Pour initialiser ce projet, pour commencer à le versionner avec git, il suffit de taper :

```
cd CalculatorProject
git init
```

5.0.2 Clone d'un projet distant

```
git clone https://github.com/ton-org/ton-projet.git
cd ton-projet
git checkout -b develop
```

Exercice

- Création d'un projet.
- clone d'un projet.

6 Travail en local

Exercice

- Initialiser un projet
- Commande `status` :
 - avant et après création ou édition d'un fichier
 - avant et après ajout d'un fichier à l'index ou à l'historique
- Vérification des différences entre les changements locaux et l'historique : commande `diff --staged`
- Commande `add`
- Commande `commit`
- Commande `log`

7 Travail avec remote

Exercice

- Création d'un projet sur GitHub
- Commande `remote`
- Mise en ligne
- Commande `push`

8 Travail en équipe

Exercice

Travail en binôme.

- Récupération du travail d'un collègue
- Commande `clone`
- Commande `pull` : `fetch+ merge`
- Gestion d'un conflit

9 Workflow

Exercice

Travail en binôme.

- Commande `branch`
- Commande `switch`
- Commande `checkout`

10 .gitignore

Le fichier `.gitignore` est un simple fichier texte placé dans le répertoire racine de votre dépôt Git. Il contient une liste de fichiers et de répertoires à ignorer, comme les fichiers temporaires, les fichiers générés par le système, les fichiers de configuration, ou ceux liés à un build.

Il permet de garder un historique propre et de ne pas intégrer dans le projet les fichiers superflus.

```
##### Build files
build/

##### Log Files
*.log

##### C++
# Prerequisites
*.d

# Compiled Object files
*.slo
*.lo
*.o
*.obj

# Precompiled Headers
*.gch
*.pch

# Compiled Dynamic libraries
*.so
*.dylib
*.dll

# Fortran module files
*.mod
*.smod
```

```
# Compiled Static libraries
*.lai
*.la
*.a
*.lib

# Executables
*.exe
*.out
*.app

#### VisualStudioCode
.vscode/*
!.vscode/settings.json
!.vscode/tasks.json
!.vscode/launch.json
!.vscode/extensions.json
*.code-workspace

# Local History for Visual Studio Code
.history/
```

11 Evaluation

Les critères suivants seront évalués :

- Le code a été rendu via un **dépôt git**
- Les commits sont **unitaires**
- Les noms des commits sont **explicites**
- Le nombre de commits est **équilibré entre les membres** de l'équipe